C-EASY-4：小应用之五子棋游戏

先按照任务提示建立起一系列源文件和头文件，这是一种模块化编程的思维。 将题目所给的一些定义和思路代码先粘贴进对应的文件中。

第一步：绘制棋盘。

先开始处理console的两个文件，console.h定义控制台相关的功能，console.c真正实现绘制棋盘。

之后在main.c的文件中先通过两个for循环对二维数组进行初始化，之后再调用console的绘制棋盘的函数，实现最终画出初始棋盘。

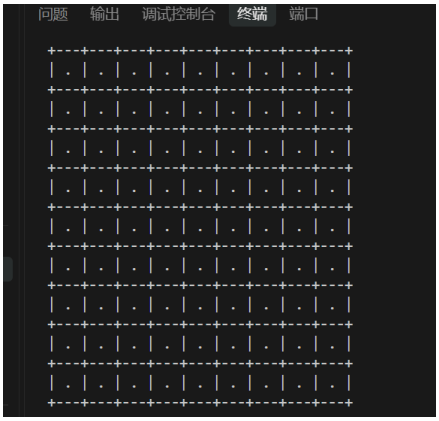
console.c中的初步函数如下：

```
#include <stdio.h>
#include "console.h"

void draw_board(const Game* game) {
    for (int col = 0; col < BOARD_SIZE; col++) {
        printf("+---");
    }
    printf("+\n");

    for (int row = 0; row < BOARD_SIZE; row++) {
        for (int col = 0; col < BOARD_SIZE; col++) {
            printf("| %c ", game->cells[row][col]);
        }
        printf("|\n");

        for (int col = 0; col < BOARD_SIZE; col++) {
            printf("+---");
        }
        printf("+\n");
    }
}
```




```
int main(){
    Game game;

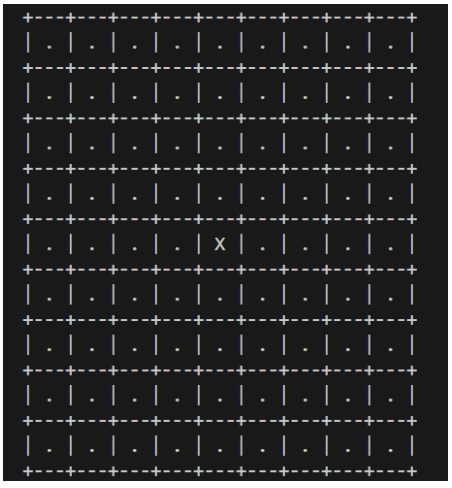
    // 初始化棋盘
    for (int r = 0; r < BOARD_SIZE; r++) {
        for (int c = 0; c < BOARD_SIZE; c++) {
            game.cells[r][c] = EMPTY_CELL;
        }
    }

    // 测试：在中间随便放一个棋子
    game.cells[4][4] = 'X';

    //这里传进去的必须要是地址
    draw_board(&game);

    return 0;
}
```

我在其中加入了一个测试x，



删掉测试x后开始第二步。

第二步：实现鼠标点击落子

初步学习了解“Windows Console API”，

- 过程理解：用 `GetStdHandle` 拿钥匙去控制台快递站，用 `ReadConsoleInput` 等包裹。拆开包裹，如果是鼠标左键按下（`FROM_LEFT_1ST_BUTTON_PRESSED`），就把坐标 `X，Y` 掏出来。
- 开始了解明白这些复杂单词是“获取鼠标光标位置”的固定写法，学会调用。

踩坑的事件		
点棋盘完全没反应，光标只变成框选	快速编辑模式下，鼠标点击被控制台当成"选中文本"，程序收不到事件	disable_quick_edit() **定义并在 main 里调用**
点一下程序立刻关闭	get_mouse_left_click 正常返回坐标后，main 直接 return 0，控制台窗口一闪而过	return 0 前加 system("pause"); 看清结果

完成后的测试代码如下:

```c
#include <stdio.h>
#include "game.h"
#include "console.h"
#include <windows.h>

void enable_mouse_input() {
    HANDLE hInput = GetStdHandle(STD_INPUT_HANDLE);
    DWORD mode;
    GetConsoleMode(hInput, &mode);

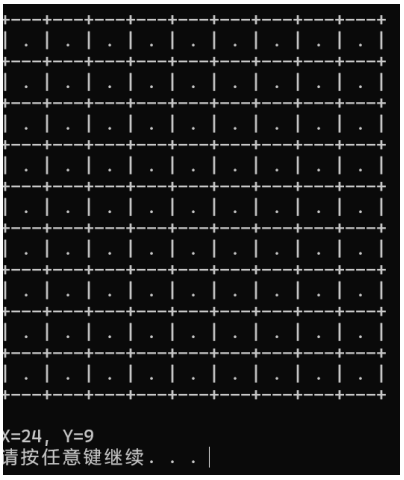
    mode &= ~ENABLE_QUICK_EDIT_MODE;  // 1. 关闭快速编辑（防止点击变成框选）
    mode |= ENABLE_EXTENDED_FLAGS;    // 2. 必须带扩展标志，上一步才生效
    mode |= ENABLE_MOUSE_INPUT;       // 3. 开启鼠标事件，ReadConsoleInput 才能收到点击
    SetConsoleMode(hInput, mode);
}
int main() {
    enable_mouse_input();
    Game game;
    for (int r = 0; r < BOARD_SIZE; r++) {
        for (int c = 0; c < BOARD_SIZE; c++) {
            game.cells[r][c] = EMPTY_CELL;
        }
    }
    draw_board(&game);

    int mouse_x, mouse_y;

    get_mouse_left_click(&mouse_x, &mouse_y);

    printf("\nX=%d, Y=%d\n", mouse_x, mouse_y);
    system("pause");   // 停住，按任意键再关窗口

    return 0;
}
```



第三步：实现循环下棋

> **棋子落下的位置和鼠标返回值的关系**
>
> 从列的角度来看 第一个棋子【0】对应1 2 3 4列 第二个棋子【1】对应 5 6 7 8列
>
> 以此类推，列数可以化作（mouse_x-1）/4
>
> 从行的角度来看 第一个棋子【0】对应1 2 行 第二个棋子【1】对应 3 4行
>
> 以此类推，行数可以化作（mouse_y-1）/2

这里有一个必须小心的坑：C 的整数除法是"向零截断"

```
(-1) / 4   ==  0      // 不是 -1!
(-5) / 4   ==  -1
```

所以绝不能让负数进去做除法——`mouse_x = 0`（棋盘左边的空白）会被算成"第 0 列"。必须先做边界检查，再做除法，顺序不能反

在`game_place`中先判断：

```
if (mouse_x < 0 || mouse_x > 36)
        return MOVE_OUT_OF_BOUNDS;
if (mouse_y < 0 || mouse_y > 18)
        return MOVE_OUT_OF_BOUNDS;
```

主循环的结构：

```
while (game.status == GAME_RUNNING) {      // 循环条件是游戏状态，不是 while(1)
    get_mouse_left_click(&mouse_x, &mouse_y);    // 阻塞等一次点击

    int row = (mouse_y - 1) / 2;
    int col = (mouse_x - 1) / 4;
    if (row < 0 || row >= BOARD_SIZE || col < 0 || col >= BOARD_SIZE)
        continue;                                  // 点在棋盘外，忽略，重新等

    game_place(&game, mouse_x, mouse_y);           // 落子
    clear_screen();
    draw_board(&game);                             // 重绘
}
```

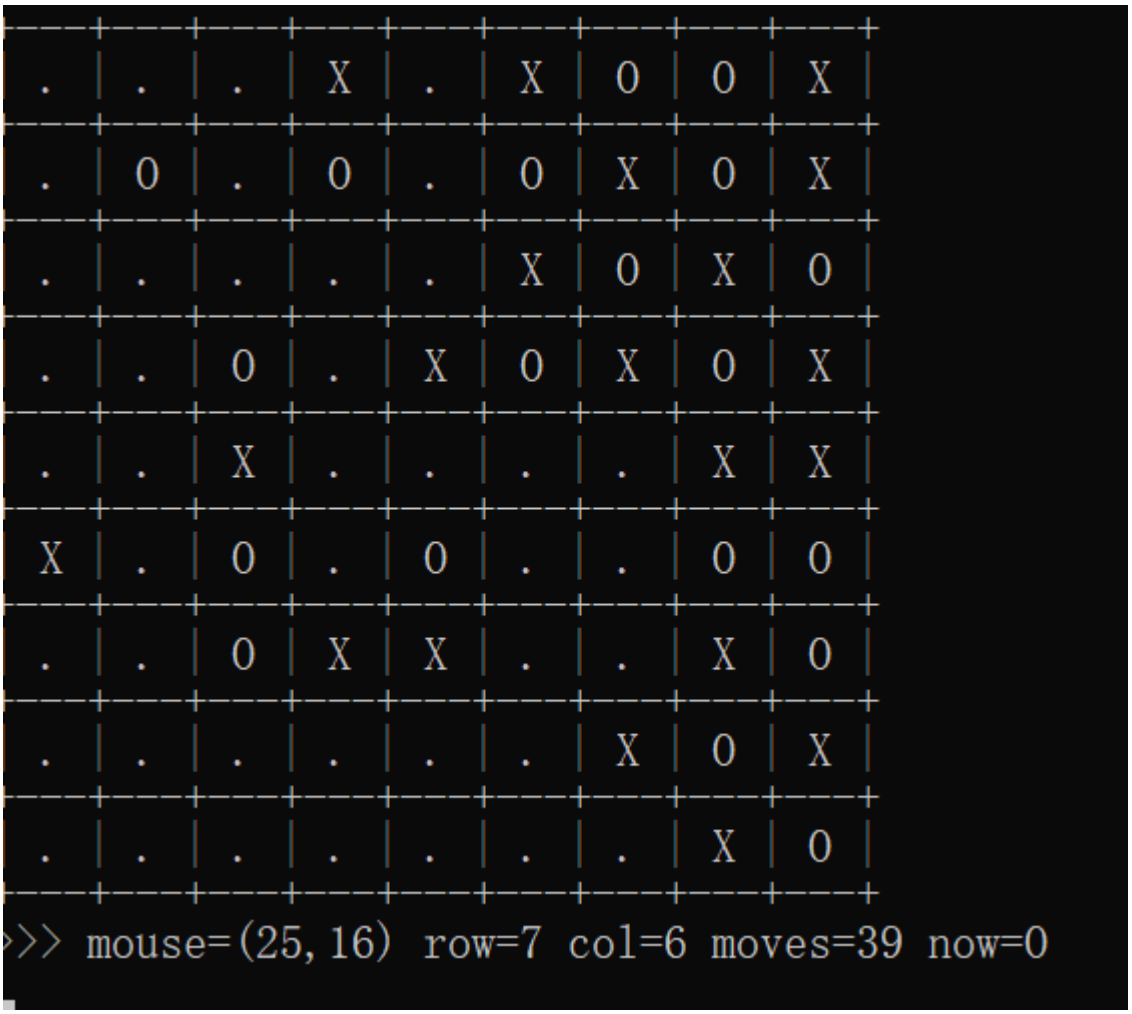
三个设计要点：

① 循环条件是`game.status == GAME_RUNNING`，不是`while(1)`。*判定出胜负后`status`变化，循环自然退出——游戏状态自己就是循环的开关，不需`break`。

② `clear_screen()` 放在重绘之前。先擦干净、再画新的。如果放在结尾，进循环前还得再清一次，容易漏。

③ `continue` 用来跳过"点在棋盘外"。 它跳回 `while` 开头重新等点击，效果就是点到棋盘外等于什么都没发生，和点到已占用格子的表现一致。

测试截图如下：



第四步：胜负判定

每次检测新落的那一颗

每次落子只可能让"经过这个格子的四条线"产生五连，其他地方的连线在之前的回合早就检查过了（如果成了，游戏早就结束了）。

所以 `is_winning_move(*game, row, col)` 接收坐标参数，专门检查**刚刚落下的那一颗**。

只用查 4 个方向，而不是 8 个

一个格子周围确实有 8 个邻居、8 个方向，但它们组成 **4 条直线**：

方向对	直线
左 ←→ 右	横
上 ←→ 下	竖
左上 ←→ 右下	主对角线 `\`
右上 ←→ 左下	副对角线 `/`

五子连珠只可能发生在一条直线上，所以只需 4 条轴。**每条轴内部"正反各数一次"，就覆盖了 8 个方向。**

用两个数组表示 4 个方向的行列增量：

```
static const int DIR_R[4] = { 0, 1, 1,  1 };   // 行增量
static const int DIR_C[4] = { 1, 0, 1, -1 };   // 列增量
```

内层循环用 `+ DIR` 数正方向、用 `- DIR` 数反方向，**一个 `d` 就覆盖了一对相反方向**。

实现细节

① `count = 1` 别忘了。调用这个函数时，那颗棋子已经写进 `cells` 了，它自己要算一个。

② 出界检查必须写在访问数组之前。

```
if (r < 0 || r >= BOARD_SIZE || c < 0 || c >= BOARD_SIZE) break;   // ← 先查
if (game->cells[r][c] != who) break;                               // ← 后访问
```

顺序反了就是灾难：数到棋盘边缘时 `r` 会变成 `-1`（读脏内存，可能崩）。

③ 用 `count >= WIN_LENGTH` 而不是 `==`。*这样长连（6 颗、7 颗）也算赢。写 `==` 的话玩家做出 6 连反而判不出胜负。

④ `count` 必须在 `for (int d...)` 循环体内声明。 放在外面的话，四个方向的计数会累加到一起，乱判。

平局判定

```
else if (game->move_count >= BOARD_SIZE * BOARD_SIZE)
    game->status = GAME_DRAW;
```

注意 `move_count` 只在真正落子成功后才 `++`（点在已占用的格子上不算）。否则平局条件会提前误触发。

第五步：最后收尾

实现控制台初始化

先定义一个`g_old_in_mode`存放控制台的原始模式。

修改一下`enable_mouse_input`的函数，增加让当前的控制台的模式得以备份到原始模式的代码。

定义一个初始化控制台的函数，设置控制台模式为原始模式。

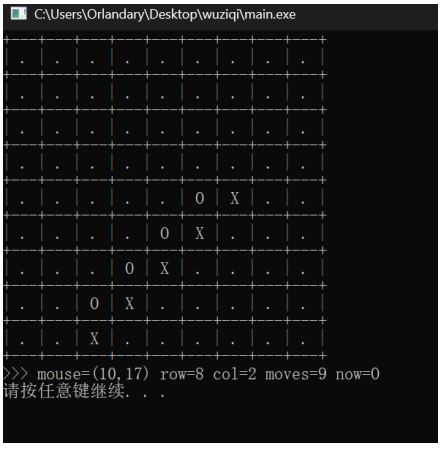
最后主函数引用。

内存管理说明

本项目未使用 `malloc` 动态内存分配，原因如下：

1. **棋盘尺寸确定**：`BOARD_SIZE` 为 `#define` 常量（9），`Game` 结构体大小在编译时即固定（约 88 字节），无需运行时决定大小；
2. **生命周期与作用域一致**：`Game` 结构体作为 `main` 函数的局部变量声明在栈上，其生命周期由作用域自动管理，`main` 返回时由编译器自动回收，不存在泄漏问题；
3. **无需跨函数持有**：所有对棋盘的访问都通过指针参数 `Game*` 在函数间传递，不需要数据活过 `main`。

最终实现图：



踩坑记录（本次调试的真实过程）

坑 1：点击第一次之后，程序再也不响应鼠标

现象： 程序能落第一颗子，之后再怎么点都没反应，像死机。

排查过程：

一开始怀疑是"忘了在循环里重画棋盘"，改成循环内重画后仍然不行。于是加调试打印，确认程序卡在 `ReadConsoleInput` 上——**第二次点击的事件根本没进来**。

根因：`system("cls")` 会启动一个 `cmd.exe` 子进程！

每回合执行一次 `system("cls")`，就是每回合启动一个子进程。子进程和主程序共用同一个控制台，会**抢走控制台的输入焦点**，导致后续的鼠标事件无法送达主程序。

解决：改用 Windows API 清屏，不启动任何子进程。

```
void clear_screen(void) {
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_SCREEN_BUFFER_INFO csbi;
    DWORD written;
    COORD home = { 0, 0 };

    GetConsoleScreenBufferInfo(hOut, &csbi);
    DWORD cells = (DWORD)csbi.dwSize.X * csbi.dwSize.Y;
    FillConsoleOutputCharacterA(hOut, ' ', cells, home, &written);   // 填空格
    FillConsoleOutputAttribute(hOut, csbi.wAttributes, cells, home, &written);
    SetConsoleCursorPosition(hOut, home);                             // 光标回左上角
}
```

经验：`system()` 这类"调用外部命令"的函数，和 `ReadConsoleInput` 这种底层输入读取不要混用。**同理，最后的"按任意键退出"也不应该用 `system("pause")`。**

坑 2：鼠标点击被静默吞掉

原因： 判定条件用了精确比较。

```
if (mer.dwButtonState == FROM_LEFT_1ST_BUTTON_PRESSED && ...)   // X 错
```

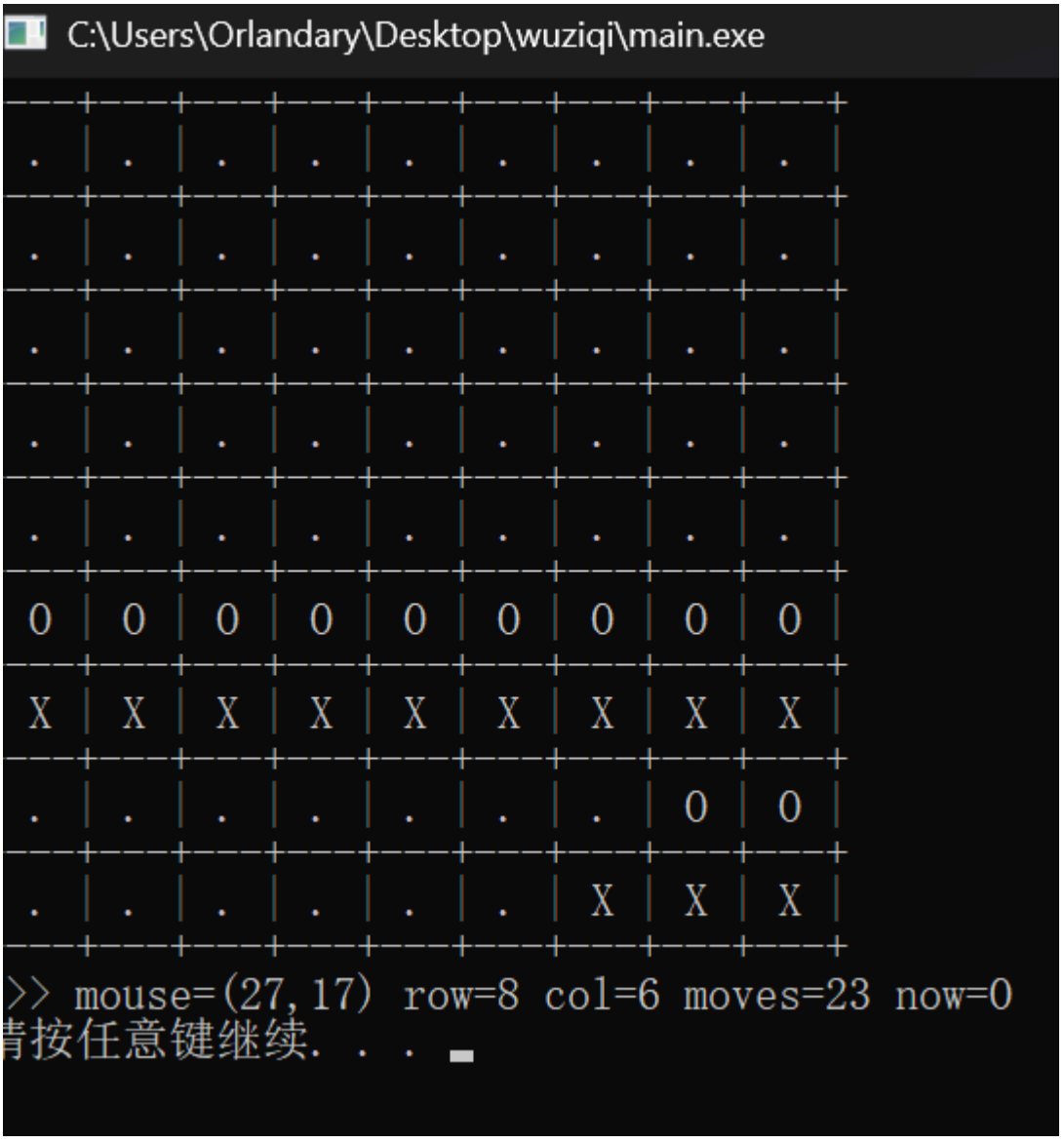
`dwButtonState` 是位掩码，可能同时有多个位。只要有别的位存在（比如右键也按着、或者双击的第二次），`==` 就不成立，**点击被静默忽略，没有任何报错**。

解决：改成按位与。

```
if ((mer.dwButtonState & FROM_LEFT_1ST_BUTTON_PRESSED) && ...)   // ✓ 对
```

坑 3：写上代码，多次尝试五连，循环都跳不出去

现象：



排查过程：

先确认 `is_winning_move` 本身没问题（逐行核对 + 单独写测试用例验证四个方向）。然后盯 `main` 里的调用：

```
game.status = check_game_status(&game, (mouse_x - 1) / 4, (mouse_y - 1) / 2);
//                                      └── 这是列 ──┘  └── 这是行 ──┘
```

根因：行列传反了。

`check_game_status(Game* game, int row, int col)` 的第一个参数是 **row**，但传进去的是 `(mouse_x-1)/4` ——那是**列**。

为什么反了会跳不出循环？ 因为 `check_game_status` 第一件事就是去读"刚落子的那一格"：

```
if (game->cells[row][col] != EMPTY_CELL && is_winning_move(game, row, col))
```

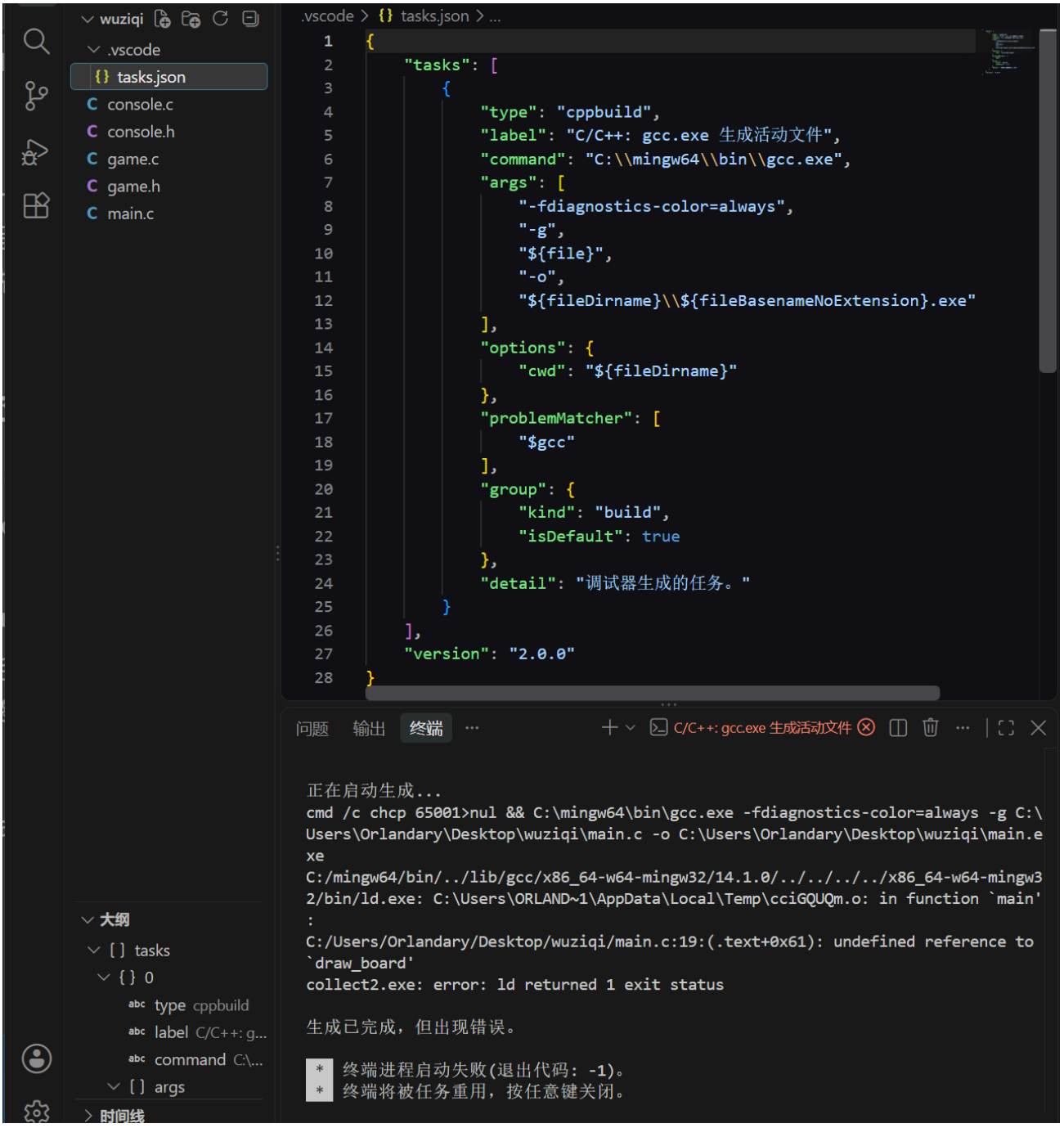
行列反了之后，它读的是**另一个格子**——通常是空格子。空格子让整个条件短路，走到 `else` 分支，`status` 被设回 `GAME_RUNNING`，循环继续。*

教训：同一个公式（`(mouse_x-1)/4`）在同一个循环里出现了三次（传给 `check_game_status`、传给 `printf`、`game_place` 内部又算一遍），只要有一处顺序写反就出问题。**根本的改进方向是"算一次、存进变量、全部复用"。**

其他学习到的知识

> **避坑指南：`#ifndef` 的作用** 用于防止头文件重复包含（防重定义报错）。每个 `.h` 文件的"标准外衣"：
>
> ```
> #ifndef GAME_H
> #define GAME_H
> // ... 具体的代码内容 ...
> #endif
> ```
>
> **原理**：第一次包含时 `GAME_H` 还没定义，`#ifndef` 条件成立，于是定义 `GAME_H` 并包含内容；第二次再包含时 `GAME_H` 已经定义，条件不成立，内容被跳过。**这样就保证内容只被包含一次。**
>
> 反例：如果 `game.h` 被 `console.h` 和 `main.c` 都 `#include` 了，没有保护的话 `Game` 结构体会被定义两次，编译直接报 `redefinition`。
>
> 命名习惯：宏名一般用**文件名全大写** + **`.` 换成 `_`**，比如 `game.h` → `GAME_H`、`console.h` → `CONSOLE_H`。

第一次尝试就失败，后来发现是 vscode 配置问题，强化了我招新第一问的感悟



在 `tasks.json` 中

- `"${file}"`：只编译当前打开的那个文件（也就是 `main.c`）。
- `"${fileDirname}\\*.c"`：编译当前文件夹下的**所有** `.c` 文件（包含了 `console.c` 和 `game.c`）。

教训：多文件项目必须编译**所有** `.c` 文件。只编译当前打开的文件，其他文件里定义的函数就找不到，报 `undefined reference`。

遇到"函数明明写了却找不到"，第一件事就是看编译命令带了几个文件。

`static` 限定内部实现

`is_winning_move` 函数、`DIR_R` / `DIR_C` 数组都加了 `static`，只在本文件可见，避免和其他文件重名冲突（否则链接时会报 `multiple definition`）。

`static` 的两个不同含义，别搞混：

用在哪	含义	一句话
函数前	**内部链接**	"这个函数/变量只有本文件能用"
函数内部的变量前	**静态存储期**	"这个变量出了函数也不销毁"

这两个用法**毫无关系**，只是碰巧共用一个关键字。

判断标准很简单：**这个函数需要写进 `.h` 文件给别人用吗？需要就不加 `static`，不需要就加。**

例如 `check_game_status` 要给 `main.c` 用，所以**不能**加 `static`；而 `is_winning_move` 只是 `game.c` 内部的小工具，就**应该**加。

好处：① 不会和别的文件里的同名函数冲突；② 相当于告诉别人"这是内部实现，别乱用"。

`const` 修饰指针参数

```
void draw_board(const Game* game);
```

这是在向调用者承诺"我只读，不改你的数据"。如果函数体里不小心写了 `game->cells[r][c] = 'X';`，**编译器会直接报错拦住你**。

什么时候加？**只要函数不修改这个结构体，就加上。**

- `draw_board`（只画）、`game_status_text`（只读状态）→ **加**
- `game_place`（要改棋盘）、`check_game_status`（要改 status）→ **不加**

Windows 变量命名习惯

前缀	类型	例子
`dw`	DWORD（32 位无符号整数）	`dwButtonState`、`dwEventFlags`、`dwSize`
`h`	HANDLE（资源句柄，本质是 `void*`）	`hInput`、`hOut`
`b`	BOOL / BYTE	`bVisible`、`bKeyDown`
`lp` / `p`	指针	`lpBuffer`、`pMode`

看懂了前缀，看到陌生变量名也能大致猜出它的类型和用途。

`DWORD` 和 `HANDLE` 到底是什么

```
typedef unsigned long DWORD;     // 就是一个 32 位无符号整数，Windows 给它起了个老名字
typedef void *HANDLE;            // 就是一个"万能指针"，代表一个系统资源的句柄
```

`DWORD` = Double Word。早期 Intel CPU 里一个"字（word）"是 16 位，"双字"就是 32 位。现在看名字有点过时，你就当它是 `unsigned long`。

为什么用**无符号**？因为控制台模式、按键状态这些都是**位掩码**（一串 0/1 开关），不需要负数。用有符号类型的话最高位会被当成符号位，位运算容易出岔子。

`HANDLE` = 门把手。你抓不住门，但抓住把手就能开门。它指向什么你不用管、也不该管（Windows 故意不让你知道内部结构），**只能原样传给 API 函数**：

```
HANDLE h = GetStdHandle(STD_INPUT_HANDLE);
printf("%d", *h);        // X 编译错误：void* 不能解引用 —— 它本来就不让你碰
```

注意：`GetStdHandle` 拿到的标准句柄不需要也不应该 `CloseHandle` —— 那是系统维护的资源，关了会导致后续输出失败。

> **为什么管它叫"字符坐标"**
>
> 在终端里，"字符大小"不是一个像素尺寸，而是**格子编号**。终端把屏幕切成等宽等高的网格，每个格子放一个字符。`printf("abc")` 不是印了三个像素宽的东西，而是**占用了 3 个格子**。
>
> 所以 `dwMousePosition.X` 返回的就是格子编号，**从 0 开始**——屏幕最左那一格是 `X = 0`。这和字体大小、窗口缩放都无关。
>
> **"字符坐标" = "第几个格子"。**

> **`%c` 和 `%s` 的区别**
>
> | 格式符 | 要什么 | 例子 |
> | --- | --- | --- |
> | `%c` | 单个字符（`char`） | `game.current_player` |
> | `%s` | 字符串指针（`char*`） | `game_status_text(&game)` |
>
> 写错会**运行时崩溃**（不是编译错误，所以更危险）——`printf` 会把一个内存地址的低位字节当成字符打印，通常是乱码，运气不好直接崩。
>
> **记忆：函数名里有 `text` 的一般返回字符串，用 `%s`。**

> **`system()` 是个危险函数**
>
> `system("cls")` 和 `system("pause")` 都会**启动一个 `cmd.exe` 子进程**。子进程与主程序共用控制台，会争抢输入焦点，和 `ReadConsoleInput` 冲突。
>
> **需要清屏 → 用 `FillConsoleOutputCharacterA` 需要等待按键 → 用 `ReadConsoleInput` 读 `KEY_EVENT`**
>
> 更一般地说：`system()` 执行的是**外部命令**，依赖运行环境，在正规项目里属于坏习惯。能不用就不用。

> **排查"没反应"类问题的通用方法**
>
> 程序"没反应"有两大类原因，**修法完全相反**，所以第一步必须分清是哪一类：
>
> | 现象 | 说明 | 方向 |
> | --- | --- | --- |
> | 加打印后，**新的一行都没出现** | 事件根本没进来 | 查输入/焦点/权限 |
> | 加打印后，**有新行但结果不对** | 事件进来了但被过滤或算错 | 查判断条件和公式 |
>
> **方法：在关键位置加一句 `printf`，把收到的原始数据打出来。** 这比盯着代码猜快十倍。
>
> 注意打印的位置——如果后面紧跟着清屏，打印会被擦掉，要放在清屏**之后**，或者临时把清屏注释掉。